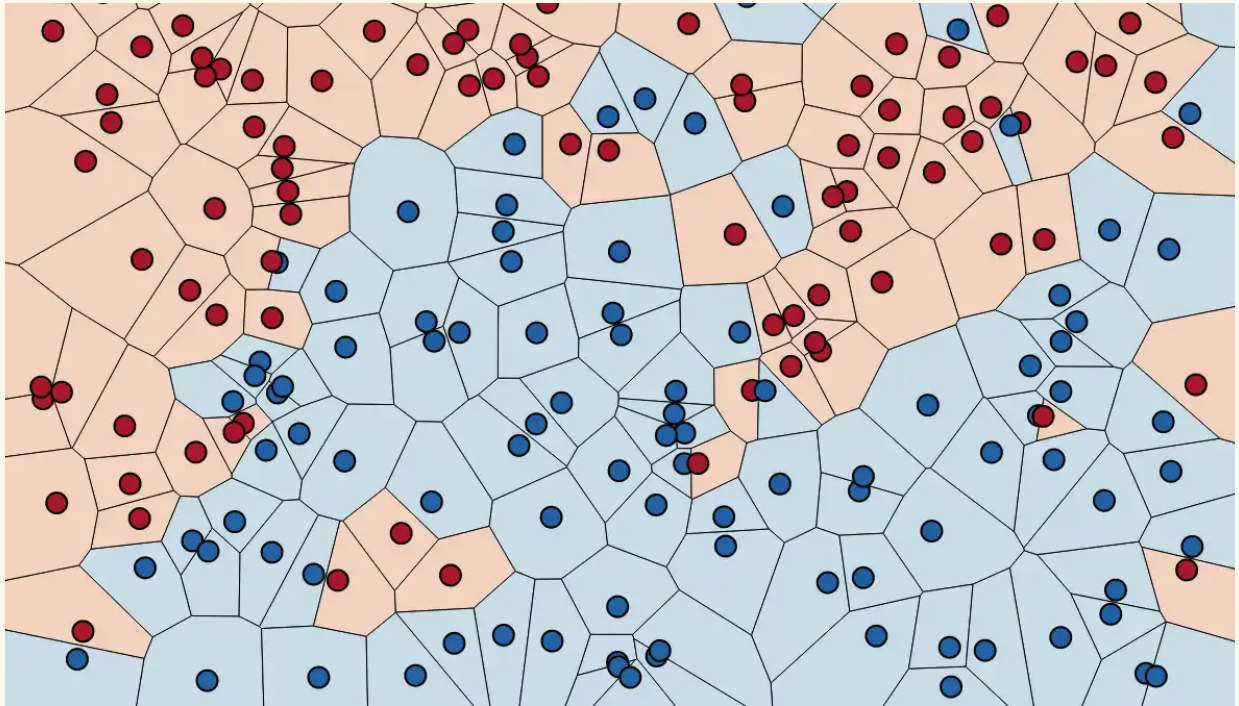


# REPORT PROGETTO



|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>RIASSUNTO.....</b>                                      | <b>4</b>  |
| Progetto Base: Sistemi di Raccomandazione.....             | 4         |
| Progetto Intermedio: Sistemi Content-Based.....            | 4         |
| Progetto Avanzato: Sentiment Analysis.....                 | 4         |
| <b>INTRODUZIONE.....</b>                                   | <b>5</b>  |
| Descrizione dei Dati.....                                  | 5         |
| Item.....                                                  | 5         |
| R-Token.....                                               | 5         |
| M-Token.....                                               | 5         |
| Composizione del DataSet.....                              | 5         |
| Analisi Esplorativa.....                                   | 5         |
| Statistiche Descrittive.....                               | 5         |
| DATI MANCANTI.....                                         | 6         |
| STATISTICHE DEL DATAFRAME.....                             | 6         |
| Statistiche dell'attributo Rating.....                     | 6         |
| RECENSIONI PER UTENTE.....                                 | 7         |
| RECENSIONI PER ITEM.....                                   | 9         |
| UTENTI E ITEM MIGLIORI.....                                | 10        |
| QQ Plot.....                                               | 11        |
| ANALISI TEMPORALE.....                                     | 11        |
| Analisi di Correlazione.....                               | 13        |
| Rating VS mean_user_rating (0.86).....                     | 14        |
| Rating VS mean_item_rating (0.43).....                     | 14        |
| <b>PROGETTO BASE.....</b>                                  | <b>15</b> |
| Identificazione della configurazione ottimale per KNN..... | 15        |
| Filtro.....                                                | 15        |
| Conversione.....                                           | 16        |
| Definizione e ricerca dei parametri.....                   | 16        |
| Creazione e addestramento del Modello KNN.....             | 17        |
| Filling della matrice di rating.....                       | 17        |
| K-Means.....                                               | 19        |
| Normalizzazione.....                                       | 19        |
| Trovare K ottimale.....                                    | 19        |
| Matrix Factorization.....                                  | 22        |
| Ricerca dei parametri ottimali.....                        | 22        |
| Addestramento e creazione del modello.....                 | 22        |
| Confronto dei risultati.....                               | 23        |
| <b>PROGETTO INTERMEDIO.....</b>                            | <b>24</b> |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| Dati.....                                                         | 24        |
| Pre-Processing del Testo (Natural Language Processing - NLP)..... | 25        |
| Bag-of-Words.....                                                 | 25        |
| Implementazione.....                                              | 25        |
| Risultato BoW.....                                                | 26        |
| Transformers.....                                                 | 26        |
| Implementazione.....                                              | 26        |
| Mean pooling.....                                                 | 26        |
| Addestramento e ottimizzazione.....                               | 27        |
| Risultati.....                                                    | 27        |
| Confronto dei Risultati.....                                      | 28        |
| <b>PROGETTO AVANZATO.....</b>                                     | <b>30</b> |
| Manipolazione dei dati.....                                       | 30        |
| Pre-Processing e Suddivisione dati.....                           | 31        |
| Bag of Words.....                                                 | 31        |
| Grid Search.....                                                  | 31        |
| Support Vector Machine.....                                       | 32        |
| Risultati Bag of Words.....                                       | 32        |
| Transformer.....                                                  | 33        |
| Support Vector Machine.....                                       | 33        |
| Risultati del Modello Transformer.....                            | 33        |
| <b>CONCLUSIONI.....</b>                                           | <b>34</b> |

## RIASSUNTO

Il set di dati con cui lavorare è relativo alle recensioni di diversi prodotti presenti su Amazon. Il nostro gruppo ha scelto il dataset **Musical Instruments**.

### Progetto Base: Sistemi di Raccomandazione

L'obiettivo è la creazione e la valutazione di 2 modelli di *collaborative filtering* per la predizione dei rating: K-Nearest Neighbors (KNN) e Singular Value Decomposition (SVD).

- L'algoritmo **KNN** ha ottenuto un Root Mean Square Error (RMSE) di **1.01**. Discreto
- L'algoritmo **SVD**, basato sulla fattorizzazione della matrice, ha mostrato una performance leggermente migliore con un RMSE di **0.9**.
- In termini di velocità, il **KNN** è risultato più rapido in fase di addestramento, mentre l'**SVD** si è dimostrato più veloce nel predire i risultati.
- Nel K-Means la scelta ottimale del numero di cluster è ricaduta su **2**, basandosi sul **Silhouette Score**. I due cluster identificati sono stati interpretati come "utenti standard" e "utenti entusiasti", sebbene questa divisione sia probabilmente influenzata da un bias introdotto dal filtraggio iniziale dei dati.

### Progetto Intermedio: Sistemi Content-Based

L'obiettivo è creare un sistema di raccomandazione content-based, utilizzando due approcci per rappresentare il testo: Bag-of-Words (BoW) e Transformers.

- Il modello basato su **Bag-of-Words (BoW)** ha ottenuto un **RMSE medio di 1.04**.
- Il modello basato su **Transformers** ha registrato un **RMSE medio di 1.06**.

### Progetto Avanzato: Sentiment Analysis

L'obiettivo finale è classificare il sentiment delle recensioni (positivo, negativo, neutro) utilizzando i modelli BoW e Transformer.

- Il modello **Bag-of-Words**, abbinato a SVM ha raggiunto un'accuratezza del **69%**.
- Il modello **Transformer**, anch'esso con SVM, ha ottenuto un'accuratezza pari al **67%**.

# INTRODUZIONE

Come già detto prima il nostro dataset è **Musical Instruments**.

## Descrizione dei Dati

| Utenti | Item | Valutazioni | R-Token | M-Token |
|--------|------|-------------|---------|---------|
| 1.8M   | 213K | 3M          | 182M    | 200M    |

### Item

Il numero degli item non è quello totale. Infatti non sono presenti tutti quei prodotti senza nessuna recensione. Inoltre alcuni item non possiedono metadati.

### R-Token

Sono il numero di tokens in ogni recensione utente. Ogni token è un'unità base del testo di una recensione (tendenzialmente una parola) e sono quindi generati partendo dal campo text e title. Per esempio la valutazione: "Great headphones, comfortable and sound is good" è suddivisa in 8 *tokens*.

Possiamo anche dire che ogni recensione/valutazione è lunga circa 60 *token*.

### M-Token

Sono tutti i tokens che includono tutti gli attributi dei metadati del prodotto (quindi anche description, price, details, etc...). Gli M-Token ci permettono di creare un sistema di raccomandazione basato sulla similarità tra i prodotti.

## Composizione del DataSet

Nel set di dati fornito siamo davanti a dei dati "multivariati", quindi abbiamo più variabili da tenere a mente.

## Analisi Esplorativa

### Statistiche Descrittive

Abbiamo eseguito subito un'analisi del nuovo dataframe per trovare delle statistiche descrittive:

## DATI MANCANTI

Hugging Face ha affermato di aver eliminato tutti i dati che non avessero valori. Quindi andiamo subito a vedere se ci sono dati mancanti.

|               |   |
|---------------|---|
| <b>userID</b> | 0 |
| <b>itemID</b> | 0 |
| <b>rating</b> | 0 |

Confermiamo subito che non ci sono dati mancanti. Non dovremo quindi eliminare nessun elemento per i passi successivi. Ad ogni modo nel codice sono state comunque prese precauzioni in caso ci fossero dati mancanti.

## STATISTICHE DEL DATAFRAME

|                                    |         |
|------------------------------------|---------|
| <b>Numero utenti unici</b>         | 1762679 |
| <b>Numero item unici</b>           | 213571  |
| <b>Numero totale di recensioni</b> | 3017439 |

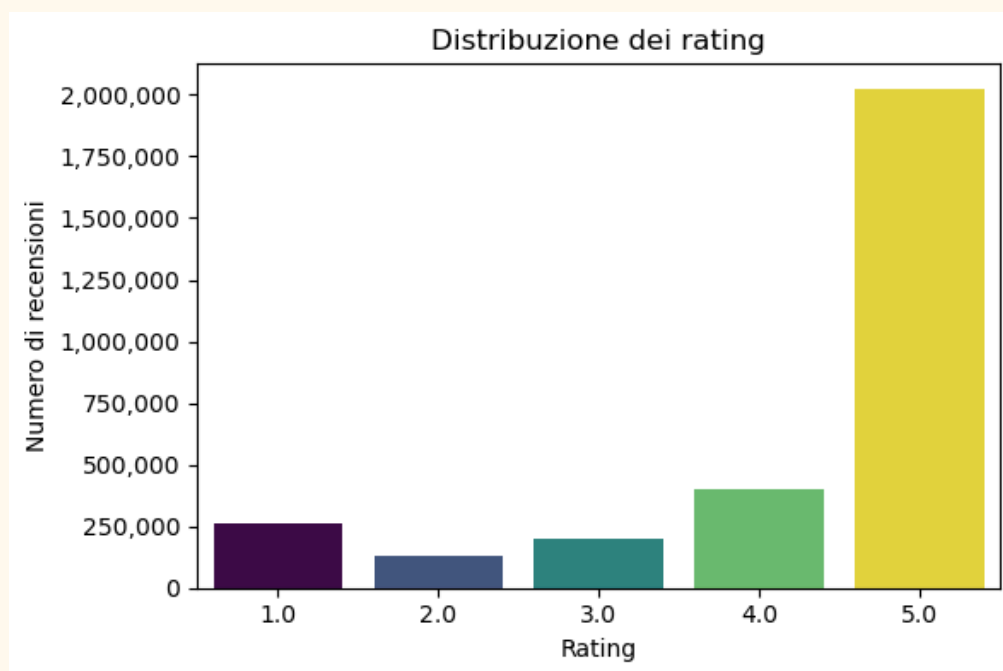
Queste prime statistiche sono coerenti con quelle fornite da Hugging Face.

## Statistiche dell'attributo Rating

|                            |           |
|----------------------------|-----------|
| <b>Totale</b>              | 3 Milioni |
| <b>Media</b>               | 4         |
| <b>Deviazione Standard</b> | 1         |
| <b>Minimo</b>              | 1         |
| <b>1° Quartile</b>         | 4         |
| <b>Mediana</b>             | 5         |
| <b>3° Quartile</b>         | 5         |
| <b>Massimo</b>             | 5         |

Notiamo subito un'altissima concentrazione di **recensioni** con **valutazione 5** in quanto mediana e massimo hanno lo stesso valore. Infatti guardando la distribuzione dei rating la barra numero 5 è visibilmente più alta, con circa **2 milioni** di recensioni con quella valutazione.

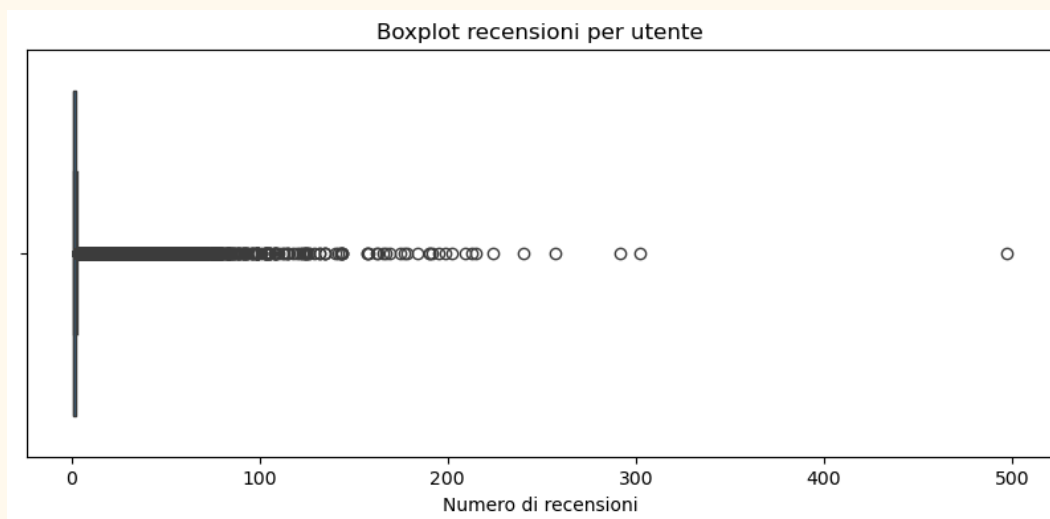
La restante parte dei rating invece hanno valori simili



### RECENSIONI PER UTENTE

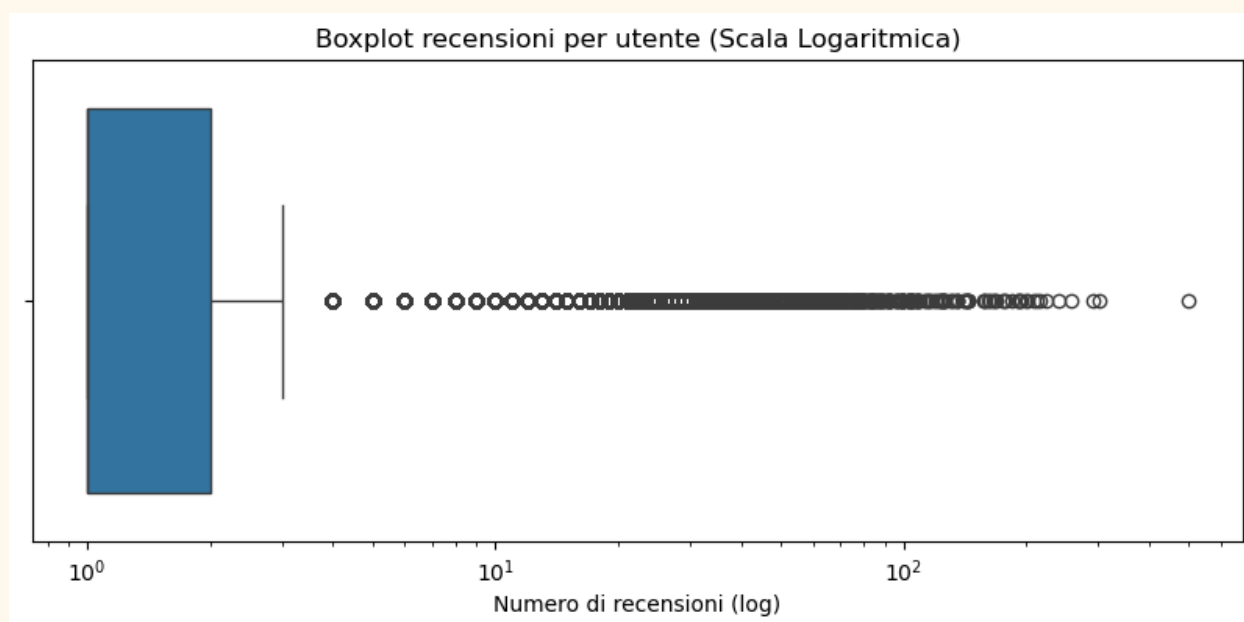
Un'altra statistica che possiamo analizzare è il **numero di recensioni** per utente:

|                             |      |
|-----------------------------|------|
| Recensioni medie per utente | 1.71 |
| Mediana                     | 1    |
| Utente con più recensioni   | 497  |
| Utente con meno recensioni  | 1    |



Osservando il box-plot capiamo molto bene che c'è una lunga coda composta da utenti che hanno fatto una sola recensione. Infatti la tabella precedente mostra che la **mediana è 1**. Questo ci fa già capire che più avanti dovremo filtrare molti dati, in quanto molti utenti (circa il 50%) avranno troppi pochi dati per poter raccomandare qualcosa (cold start problem).

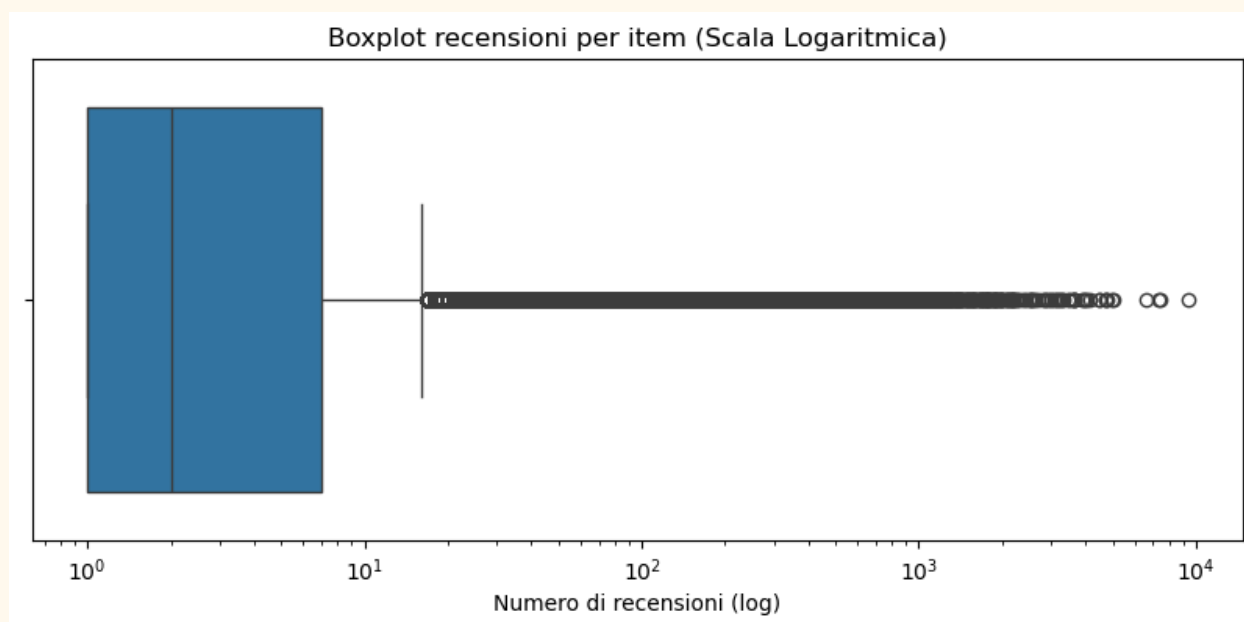
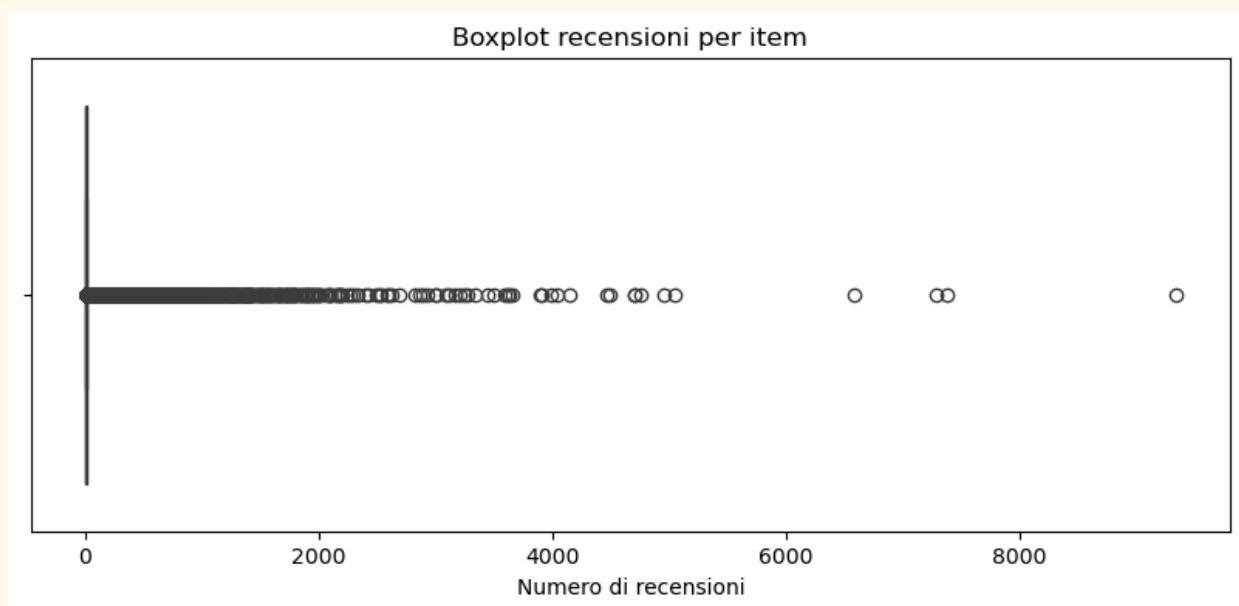
Proviamo anche ad usare una scala logaritmica, per una più facile visualizzazione:





### RECENSIONI PER ITEM

|                                  |       |
|----------------------------------|-------|
| <b>Recensioni medie per item</b> | 14.13 |
| <b>Mediana</b>                   | 2     |
| <b>Item con più recensioni</b>   | 9334  |
| <b>Item con meno recensioni</b>  | 1     |



## UTENTI E ITEM MIGLIORI

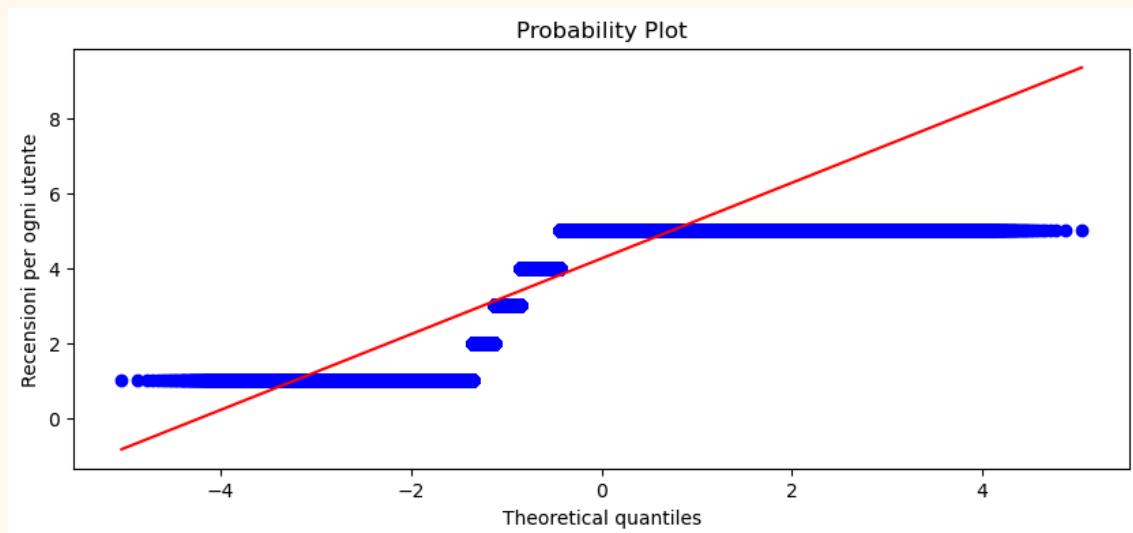
I 5 utenti più attivi:

| <i>Utente</i>   | <i>Numero di recensioni</i> | <i>Numero di acquisti verificati</i> |
|-----------------|-----------------------------|--------------------------------------|
| <b>AG3S4...</b> | 497                         | 200                                  |
| <b>AEYOY...</b> | 302                         | 51                                   |
| <b>AF7CC...</b> | 292                         | 286                                  |
| <b>AH4GZ...</b> | 257                         | 237                                  |
| <b>AHGDG...</b> | 240                         | 192                                  |

I 5 item più popolari

| <i>Item</i>                                    | <i>Numero di recensioni</i> | <i>Prezzo (in \$)</i> | <i>Valutazione Media</i> |
|------------------------------------------------|-----------------------------|-----------------------|--------------------------|
| <b>Cavo amplificatore per basso e chitarra</b> | 9334                        | 14.3                  | 4.8                      |
| <b>Microfono karaoke Bluetooth senza fili</b>  | 7380                        | 30                    | 4.4                      |
| <b>Microfono USB per PC</b>                    | 7285                        | 130                   | 4.7                      |
| <b>Pedale delay per chitarra</b>               | 6583                        | 40                    | 4.4                      |
| <b>Supporto per microfono</b>                  | 5047                        | 17                    | 4.3                      |

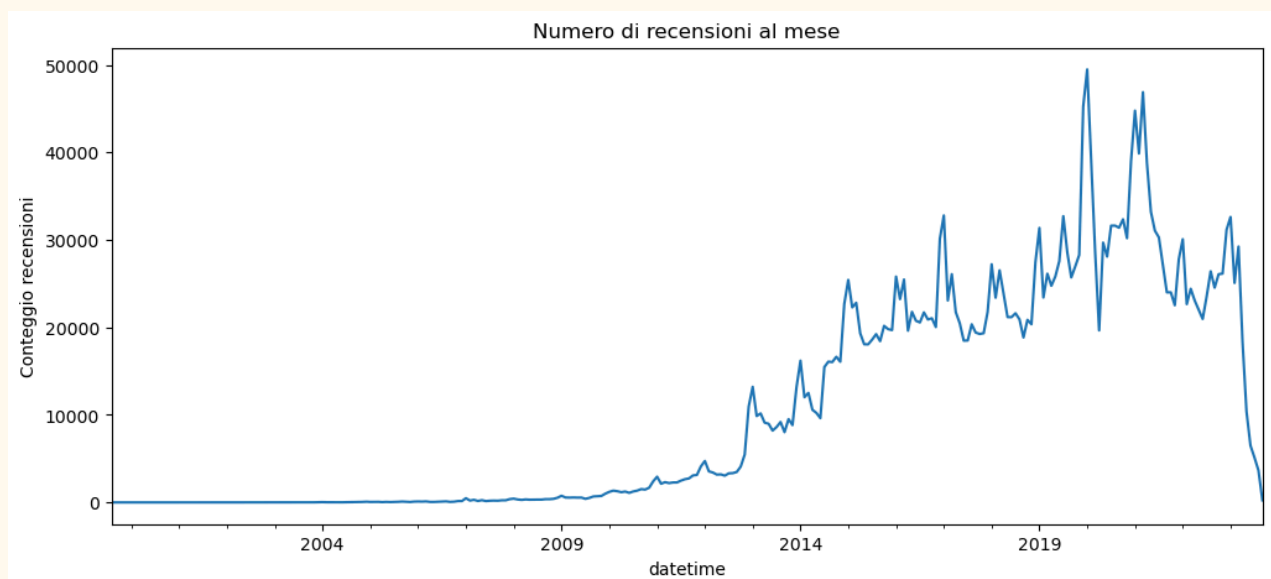
## QQ Plot



Le recensioni chiaramente **non seguono una distribuzione normale**, ma questo non è importante in quanto KNN e SVD non necessitano dell'assunzione di normalità. Quindi possiamo tranquillamente usare i dati per creare il nostro sistema di raccomandazione senza fare nessuna modifica.

## ANALISI TEMPORALE

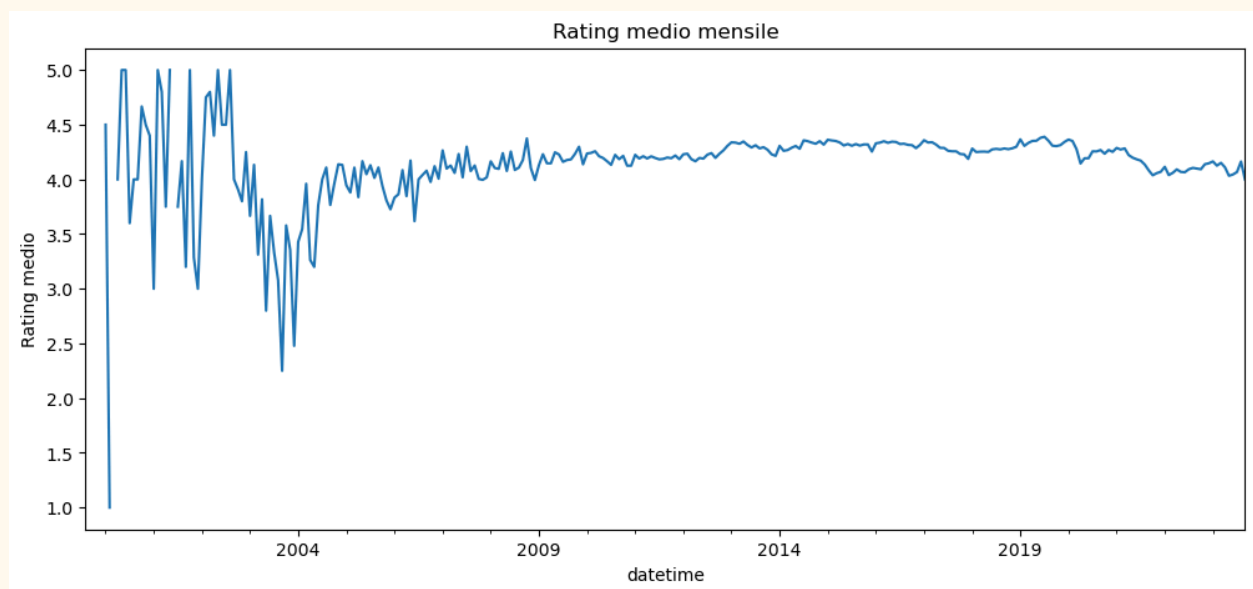
Quante recensioni vengono scritte ogni anno? Per rispondere a questa domanda possiamo visualizzare la colonna *timestamp* e contare il numero di recensioni nel tempo.



Notiamo subito che verso il **2023-2024** le recensioni crollano. Questo è sicuramente dovuto al periodo in cui è stato creato il dataset, che non ha giustamente contato tutte le nuove recensioni. Non va assolutamente interpretato come un crollo delle vendite di Amazon.

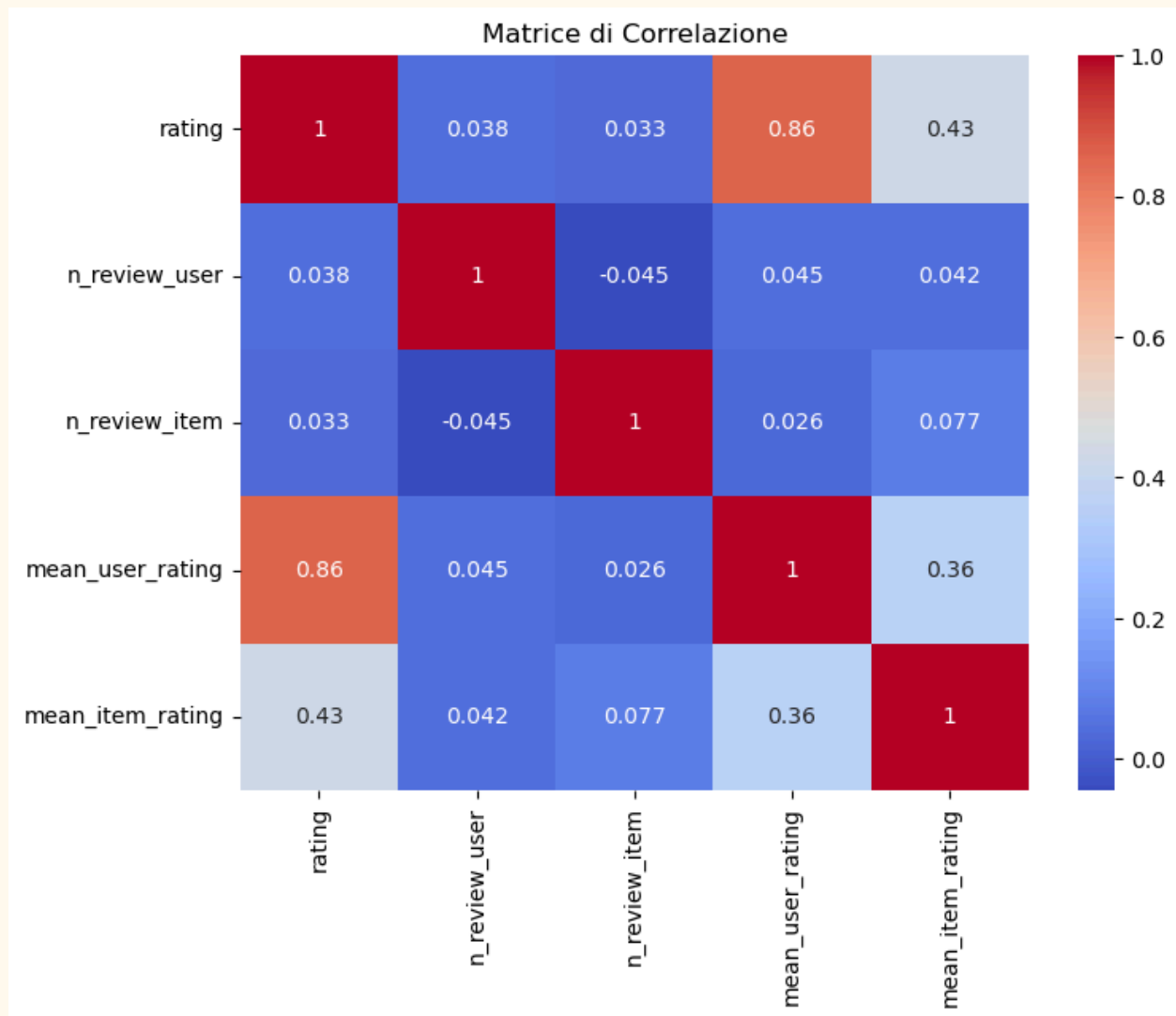
Questo non è il corso adatto per fare questo tipo di osservazione ma dall'immagine si può notare che il grafico presenta sia un **trend** sia una **seasonality** (black friday, saldi natalizi).

I prodotti tendono a ricevere voti più alti o più bassi con il passare del tempo? Questa è una domanda a cui si risponde guardando la media delle recensioni nel tempo.



Notiamo subito che dal **2007** in poi la **media** delle recensioni si è mantenuta **stabile**. Il periodo precedente presenta delle oscillazioni perché probabilmente ci sono molti meno dati in quel periodo.

## Analisi di Correlazione



Eseguendo l'analisi di correlazione notiamo subito alcune cose:

- C'è una **correlazione moderata** (0.36) tra la **media delle recensioni di un utente** e la **valutazione media dei prodotti che recensisce**. Un utente che è generalmente "alto" con le sue recensioni tende a scegliere prodotti che sono, in media, apprezzati da tutti (con delle recensioni alte).
- La singola recensione che un utente dà è quasi indipendente da quante recensioni ha già lasciato (0.038). Un **"recensore abituale"** non è propenso a dare un voto alto o basso a una specifica recensione rispetto a un utente che recensisce raramente.

- Non c'è una relazione significativa tra il numero di recensioni di un prodotto e il suo voto medio (0.077). Un prodotto **popolare non** è necessariamente un prodotto con **recensioni alte**.
- Non sono presenti correlazioni negative significative.

### Rating VS mean\_user\_rating (0.86)

La media delle recensioni che ha dato un utente è **fortemente correlata** con la recensione singola, quindi la singola recensione di un utente è un buon predittore per tutte le altre sue recensioni.

Questo risultato ci permette di riflettere molto. Se prendiamo un utente a caso e sappiamo la media delle sue recensioni, possiamo prevedere in maniera sufficientemente sicura la sua **prossima recensione**.

Questa intuizione si collega al *filtraggio collaborativo basato su utente*. Infatti l'idea è trovare un utente B che recensisce i prodotti similmente ad un utente A. Quando lo troviamo possiamo usare le recensioni di B per prevedere le recensioni di A.

### Rating VS mean\_item\_rating (0.43)

La **singola recensione** di un utente è un discreto **predittore della valutazione media** di un prodotto.

Anche questo risultato è interessante, possiamo infatti dire che se un prodotto ha delle valutazioni molto alte, allora in linea generale piacerà alla maggior parte degli utenti (effetto saggezza della folla).

Questo principio è fondamentale quando si utilizza l'*item-based collaborative filtering*, che crea un sistema di raccomandazione basato sulla **similarità** dei prodotti. Si trova prima un prodotto A e un prodotto B che sono molto simili. Se alle persone che è piaciuto il prodotto A hanno recensito bene anche il prodotto B, allora il sistema **proporrà** all'utente il prodotto B.

Successivamente, durante la scelta della modalità di applicazione del Collaborative Filtering implementeremo entrambi i casi: **user-based** e **item-based**.

## PROGETTO BASE

L'obiettivo del progetto base è quello di creare **due modelli di predizione** (KNN e SVD) e verificare le loro performance.

**Collaborative Filtering** (Filtro Collaborativo): È la tecnica usata qui. L'idea è di fare raccomandazioni a un utente basandosi sui **gusti di utenti simili** a lui (approccio user-based) oppure raccomandando **item simili** a quelli che ha già apprezzato in passato (approccio item-based).

### Identificazione della configurazione ottimale per KNN

In primis va introdotto cosa è **il KNN**: è un **algoritmo** di machine learning supervisionato. Nel machine learning supervisionato, l'algoritmo impara da un dataset di training che include sia le "feature" (le caratteristiche dei dati di input) sia le "label" (etichette, classi).

L'obiettivo è che l'algoritmo impari una mappatura tra le feature e le label, in modo da poter prevedere le label per nuovi dati non visti prima.

Il suo funzionamento si basa su questi **passaggi**:

1. **Memorizza** i dati (fase di training).
2. Cerca poi di trovare delle **somiglianze** tra i dati, lo può fare in diversi modi.
3. Dopo aver trovato tutte le **distanze euclidee** comincia a identificare i K più vicini del nuovo punto. K è un iper-parametro che viene scelto durante la scrittura del codice

### Filtro

Inizialmente puliamo e filtriamo il dataframe prendendo solo gli utenti che hanno fatto più di **30 recensioni** e solamente gli item che hanno ricevuto almeno **10 recensioni**.

Facciamo questo perché come abbiamo visto nell'esplorazione dei dati la maggior parte degli utenti ha dato solo una recensione. Il KNN è un algoritmo **poco efficiente** dal punto di vista computazionale. Infatti ad ogni predizione di un nuovo punto il KNN deve calcolare la distanza da tutti gli altri punti presenti nel dataset. Se il **dataset iniziale è molto grande** e con dati "poco rilevanti" allora stiamo sprecando risorse computazionali e tempo per nulla.

## Conversione

Poi cominciamo ad usare la **libreria “surprise”** che ci permette di usare molti strumenti. Però i dataframe di pandas hanno un formato differente e devono essere adattati. Quindi “diciamo a Surprise” che la scala di valutazione va da 1 a 5 e poi convertiamo il dataframe nel formato di Surprise.

## Definizione e ricerca dei parametri

Ora dobbiamo testare varie configurazioni. La configurazione che produrrà il modello migliore verrà scelta e applicata per predire le recensioni degli utenti.

- Proviamo a testare prima con **K=10** vicini, poi **K=20** e poi con **K=30**
- Testiamo diverse formule: **cosine**, **pearson**, **msd** per calcolare quando 2 utenti/item sono simili
- Proviamo sia l’approccio **User-Based** e sia l’approccio **Item-Based**

In totale testeremo 18 combinazioni diverse. Per valutare i vari modelli useremo le metriche MSE e RMSE, quest’ultimo misura la differenza media tra i rating predetti e i rating reali. Un RMSE più basso indica un modello migliore.

Inoltre applichiamo  $cv=5$ , ovvero la **5-fold cross-validation**.

- Il dataset viene diviso in 5 parti
- Viene addestrato sulle prime 4 e testato sull’ultima
- Poi viene addestrato sulle parti 1,2,3,5 e testato sulla 4° parte, etc...
- Viene fatta la media dei 5 risultati
- Questo processo viene fatto per tutte le 18 combinazioni

La configurazione ottimale trovata dalla gridSearchCV è:

|                   |            |
|-------------------|------------|
| <b>RMSE</b>       | 1.01       |
| <b>MSE</b>        | 1.02       |
| <b>K (vicini)</b> | 10         |
| <b>Metrica</b>    | pearson    |
| <b>Approccio</b>  | Item-Based |



## Creazione e addestramento del Modello KNN

Dopo aver identificato la migliore configurazione possibile, ora la usiamo per creare e addestrare il modello.

- creiamo **un trainset nuovo**
- creiamo una **nuova istanza** dell'algoritmo KNNBasic, ma questa volta gli forniamo direttamente i **parametri migliori**
- Infine lo **addestriamo** sul trainset creato poco prima.

Valutiamo l'algoritmo con la cross-validation dandogli in pasto l'intero dataset filtrato.

|                               | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 | Media | Dev. Std. |
|-------------------------------|--------|--------|--------|--------|--------|-------|-----------|
| <b>MSE</b>                    | 0.95   | 1.02   | 1.01   | 1.01   | 1.05   | 1.01  | 0.03      |
| <b>RMSE</b>                   | 0.97   | 1.01   | 1      | 1      | 1.02   | 1     | 0.01      |
| <b>Tempo di addestramento</b> | 0.11   | 0.11   | 0.11   | 0.12   | 0.12   | 0.11  | 0         |
| <b>Tempo di testing</b>       | 0.08   | 0.08   | 0.08   | 0.08   | 0.08   | 0.08  | 0         |

**RMSE Medio** → 1.01 : Un RMSE sotto 1 è generalmente considerato un buon risultato.

**RMSE Std** → 0.01: Una deviazione standard bassa come questa è un ottimo segno. Significa che la performance del modello è stata stabile e consistente attraverso i diversi sottoinsiemi di dati. Un'alta deviazione standard indicherebbe che il modello è molto sensibile al set di dati specifico su cui viene addestrato.

## Filling della matrice di rating

Procediamo ad effettuare il **filling** della matrice di rating con la configurazione ottimale. Il nostro nuovo algoritmo opportunamente addestrato riempirà il dataset filtrato, facendo in modo che tutte le righe NaN abbiano finalmente un valore.

In sintesi passiamo da una **matrice sparsa** (piena di parti vuote) ad una **matrice densa**.

In pratica iteriamo prima su ogni riga e poi su ogni colonna della matrice. In questo modo visitiamo ogni singola cella della matrice. Ad ogni iterazione:

- Controlliamo se la cella è vuota (NaN), se è già riempita passiamo alla prossima
- Altrimenti richiamiamo `algo.predict(uid=user_id, iid=item_id)` che predice un voto (.est)
- Riempiamo la cella con il nuovo voto

Questa è una delle parti più inefficienti del nostro algoritmo, in quanto più la matrice è grande più l'algoritmo richiede potenza computazionale. Per questo motivo abbiamo creato la matrice con il dataframe filtrato.

Vediamo un “prima e dopo” della matrice:

|                               | itemID | 1423414357 | B00005ML71 | B0002CZTIO | B0002CZVWS | B0002D01K4 | B0002D01KO | B0002D0CCQ | B0002D0CEO |
|-------------------------------|--------|------------|------------|------------|------------|------------|------------|------------|------------|
|                               | userID |            |            |            |            |            |            |            |            |
| AE23LDQTB7L76AP6E6WPBFVYL5DA  |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| AE23WLBRYKEC67DM43M6E2M7GPGQ  |        | 5.0        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| AE23ZFVUOMPKR57BVSXWV34QLMVA  |        | NaN        | NaN        | NaN        | 5.0        | NaN        | NaN        | NaN        | NaN        |
| AE27PVJJOEVGOHYF5WOXQDZ5NIULA |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| AE2AII7ZOEPCPPFDPF33HPSNBA PQ |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| ---                           | ---    | ---        | ---        | ---        | ---        | ---        | ---        | ---        | ---        |
| AHZNF3H5YLL623I5C6PO3TVZFXSQ  |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| AHZPDHP2UGRX5WII TRFMSKS2WRXA |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| AHZQPH7HHSWLUJQFWEQ54NNKN6A   |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| AHZT6MVWNF4GG6FISMZMORKZKK4A  |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |
| AHZYZ28UDD7WAJPW5G6K2DK5LYPO  |        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        | NaN        |

[illegible]

## K-Means

Ora che abbiamo una matrice densa piena di dati possiamo cominciare a cercare di suddividere gli **utenti in classi (clusters)**. Per farlo ci serviamo dell'algoritmo K-Means con cosine similarity.

### Normalizzazione

Prima di tutto normalizziamo la matrice, per usare **K-Means** con la similarità **cosenica**. Infatti K-Means usa di base la distanza euclidea, normalizzando la matrice invece portiamo tutti gli elementi della matrice ad avere la stessa lunghezza.

Il motivo dietro questa scelta è abbastanza ovvio, la **distanza euclidea** misura la distanza tra i punti in base alla "lunghezza" o "distanza assoluta". Questo approccio risulta sbagliato in un sistema di raccomandazione, la cosine similarity infatti misura la **similarità tra i punti in base alla "direzione"**, che riassume meglio la similarità di due utenti/item.

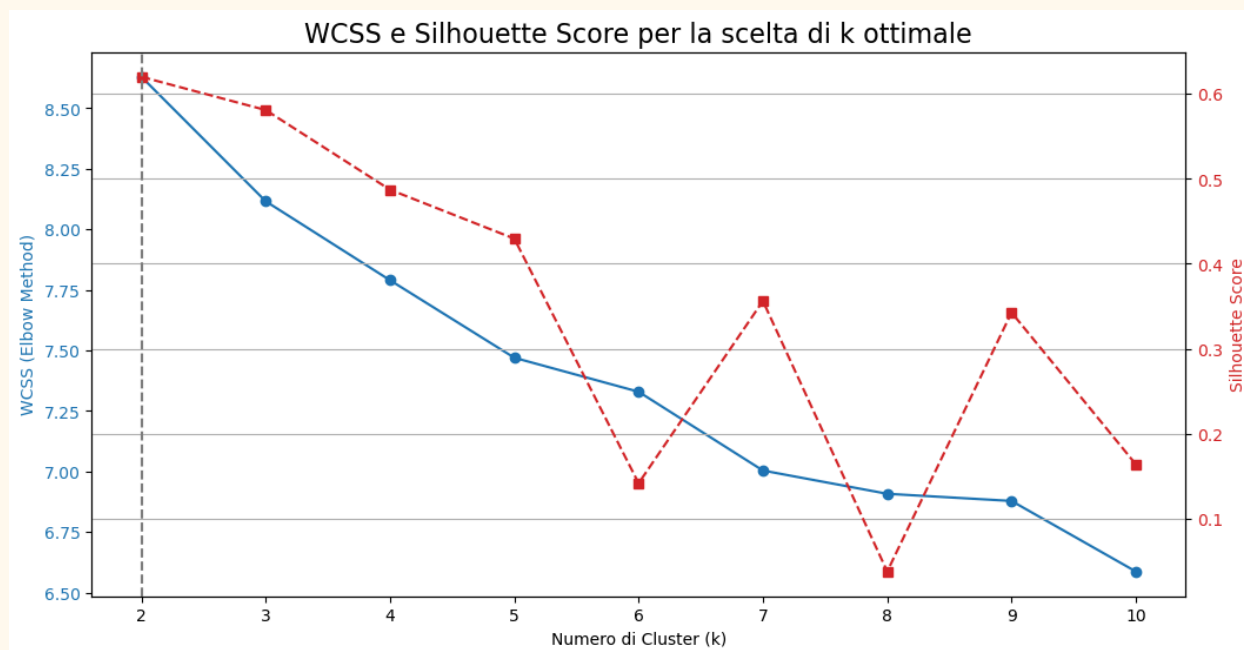
In questo modo raccomandiamo l'item che somiglia di più agli item che ha comprato l'utente, invece che l'item con la valutazione più simile a quelle che ha dato l'utente.

### Trovare K ottimale

Ora come avevamo fatto per KNNBasic dobbiamo trovare il K ottimale, che questa volta però ha un significato diverso. Il K infatti rappresenta il numero di **classi** (cluster) in cui dobbiamo dividere i nostri dati.

Per farlo testiamo diversi valori che K potrebbe assumere (2, 3, ..., 10). Per misurare le **performance** ci serviamo di **2 metriche**:

1. **WCSS** = Un WCSS basso significa che i cluster sono "densi" e compatti. Si cerca il punto in cui l'aggiunta di un nuovo cluster non causa più una diminuzione significativa del WCS, questo punto assomiglia a un "gomito" nel grafico.
2. **Silhouette Score** = Un valore vicino a 1 indica che il punto è ben inserito nel suo cluster e lontano dagli altri. Un valore vicino a 0 indica che il punto è sul confine tra due cluster. Un valore negativo indica che il punto potrebbe essere stato assegnato al cluster sbagliato.



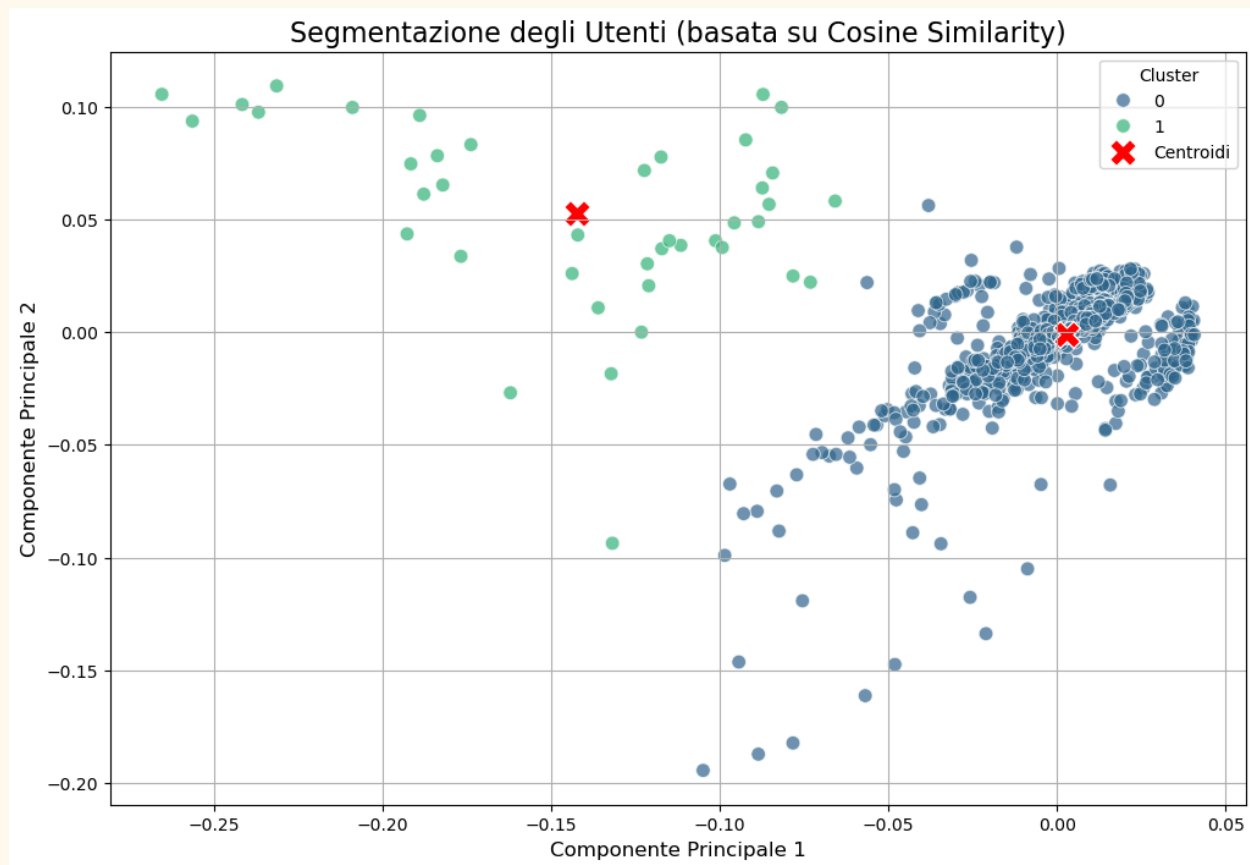
Il picco più alto per la Silhouette Score è 2, per la WCSS invece il gomito si forma intorno a  $k=4$  o  $k=5$ .

Sta di fatto però che la silhouette score è una metrica più affidabile, dunque scegliamo di implementare solo **2 cluster** (anche perché i cluster formati sono più omogenei). Applichiamo l'algoritmo con il  **$k=2$**  e lo visualizziamo immediatamente usando PCA.

| Cluster | Utenti per Cluster |
|---------|--------------------|
| 0       | 2036               |
| 1       | 41                 |

| Cluster | Media | Dev. Std | Minimo | 25% | Mediana | 75% | Massimo |
|---------|-------|----------|--------|-----|---------|-----|---------|
| 0       | 4.5   | 0.9      | 1      | 4   | 5       | 5   | 5       |
| 1       | 3.7   | 1.3      | 1      | 3   | 4       | 5   | 5       |

I risultati sono piuttosto scadenti.



I cluster sembrano essere divisi

- **Cluster 0** = utenti standard con una media delle recensioni “normale”
- **Cluster 1** = utenti entusiasti con una media molto alta

Il problema di questa divisione nasce soprattutto dal filtraggio iniziale, che ha sicuramente introdotto **un bias**. Scegliendo utenti con tot. recensioni abbiamo eliminato tutto il resto.

Persino con  $k=3$  il risultato cambia poco:

| Cluster | Utenti per Cluster | Media | Dev. Std |
|---------|--------------------|-------|----------|
| 0       | 1944               | 4.5   | 0.9      |
| 1       | 95                 | 3.78  | 1.4      |
| 2       | 38                 | 3.77  | 1.3      |

Il cluster 0 continua ad essere gigantesco ed a rappresentare gli utenti che fanno tante recensioni e con valutazioni molto alte.

## Matrix Factorization

Eseguiamo ora un filling della matrice di rating attraverso Matrix Factorization in aggiunta a K-NN. Ma cosa cambia dal KNN?

La Matrix Factorization si basa sull'esistenza di “**caratteristiche nascoste**” che spiegano le similarità e le preferenze degli utenti. Nella pratica si semplifica la matrice sparsa iniziale con 2 matrici più piccole, una per gli utenti e una per gli item.

La predizione per un utente-item è data dal **prodotto scalare** dei vettori latenti delle 2 matrici.

### Ricerca dei parametri ottimali

In primis ottimizziamo gli iperparametri con la **gridSearchCV**, proprio come avevamo fatto per il KNN. Questa volta però selezioniamo dei parametri diversi:

- **n\_epochs** = Controlla per quante volte l'algoritmo esamina l'intero dataset di training.
- **lr\_all** = è il valore che controlla la dimensione dei vettori latenti ad ogni iterazione
- **reg\_all** = serve a prevenire l'overfitting penalizzando i fattori latenti a cui è stata assegnata troppa importanza.

Cerchiamo quindi tutte le combinazioni possibili.

|                                |               |               |               |
|--------------------------------|---------------|---------------|---------------|
| <b>Miglior RMSE</b>            | 0.9           |               |               |
| <b>Migliore Configurazione</b> | n_epochs = 20 | lr_all = 0.01 | reg_all = 0.1 |

### Addestramento e creazione del modello

Una volta trovati i **parametri migliori**, li usiamo per **addestrare** un nuovo modello SVD. In questo caso creiamo un trainset con tutti i dati.

Infine **riempiamo** la **matrice** con lo stesso processo con cui l'avevamo riempita per il KNN.

Al termine di questo processo, la **svd\_filled\_matrix** conterrà una versione "densa" della matrice originale sparsa, dove tutti i valori mancanti sono stati sostituiti dalle predizioni del modello SVD.

## Confronto dei risultati

|            | MSE  | RMSE | Tempo di addestramento | Tempo di predizione |
|------------|------|------|------------------------|---------------------|
| <b>SVD</b> | 0.81 | 0.9  | 0.16                   | 0.04                |
| <b>KNN</b> | 1.02 | 1.01 | 0.10                   | 0.07                |

SVD risulta chiaramente il vincitore con un tasso di errore più basso di KNN. Inoltre vanno fatte 2 ulteriori valutazioni:

1. **La velocità di addestramento** → KNN è estremamente **più veloce da addestrare**, nonostante poi performi peggio nelle predizioni. Questo è spiegato dal fatto che SVD deve **costantemente aggiornare** i vettori latenti per evitare overfitting e underfitting.
2. **La velocità di predizione** → SVD è **più veloce a predire**, in quanto deve solamente recuperare i valori dalle 2 matrici (utente e item) e calcolare il prodotto scalare. KNN invece effettua il calcolo proprio durante la predizione cercando i vicini più simili.

La differenza però **non è** estremamente **marcata** e il tasso di errore è generalmente ancora troppo alto. Quindi possiamo concludere che nessuno dei 2 modelli sia capace di fare previsioni accurate.

Sicuramente con un miglior settaggio degli iperparametri e un dataset filtrato in maniera migliore il risultato cambierebbe.

## PROGETTO INTERMEDIO

L'obiettivo è la creazione di un sistema di raccomandazione di tipo content based. L'obiettivo è **prevedere il rating** che un utente darebbe a un prodotto basandosi esclusivamente sulle **caratteristiche testuali** (titolo e descrizione) del prodotto. Il progetto sviluppa ed analizza due approcci diversi per **rappresentare** il contenuto testuale:

1. **Bag Of Words**: Tecnica classica e semplice di vettorizzazione del testo.
2. **Transformers**: Un approccio moderno e semanticamente ricco basato su un modello neurale per-addestrato.

Infine, per entrambi gli approcci, viene utilizzato un regressore **K-Nearest Neighbors (KNN)** per predire i rating.

### Dati

È stata applicata una strategia di **filtraggio** al dataframe per incrementare l'affidabilità del modello, riducendo la sparsità e selezionando solo utenti e prodotti con **un numero adeguato** di interazioni.

Rispetto al progetto base, il filtro attuale è significativamente **più selettivo**. Seleziona solo gli **utenti** con almeno **50 recensioni** e **item** con almeno **15 recensioni**.

Ciò è dovuto alla necessità di **contenere i costi computazionali** richiesti dai nuovi algoritmi da allenare.

|                                                                    |         |
|--------------------------------------------------------------------|---------|
| Numero di recensioni totali prima del filtro                       | 3017439 |
| Numero di recensioni totali dopo il filtro (togliendo i duplicati) | 3271    |
| Numero di utenti unici                                             | 491     |
| Numero di item unici                                               | 146     |



## Pre-Processing del Testo (Natural Language Processing - NLP)

Andiamo a trasformare il **testo grezzo** in un formato **pulito e standardizzato** per l'analisi/addestramento di modelli di machine learning. Il testo grezzo contiene elementi (punteggiatura, maiuscole, parole comuni) che non portano significato semantico ma **aumentano la complessità** del vocabolario. Quindi il processo di **normalizzazione del testo**, va a **ridurre la dimensionalità** del vocabolario e raggruppa parole con lo stesso significato di base, aiutando il modello a identificare i concetti chiave.

### Bag-of-Words

Questo approccio rappresenta un **testo** come un **vettore numerico**. La lunghezza del vettore è pari alla **dimensione** del **vocabolario globale**, ogni elemento del vettore indica la frequenza (o la presenza/assenza) di una parola del vocabolario nel testo. Il **limite** di Bow è che **perde** l'ordine delle parole e il contesto semantico: ad esempio: "This is a good day" vs. "Is this a good day" sarà rappresentato in modo identico anche se il loro significato è diverso.

### Implementazione

L'approccio si basa sulla creazione di un modello di raccomandazione **personalizzato per ogni singolo utente**. Invece di un unico modello "uno per tutti", il sistema **impara le preferenze** di ciascun utente analizzando **il contenuto degli item** che ha già valutato. Il processo si articola in 2 fasi principali.

1. Viene creato un **vocabolario** da 5000 features, ovvero dalle 5000 parole più comuni.
2. Ogni articolo viene poi **trasformato** in un **vettore numerico** dove ogni numero indica quante volte una parola del vocabolario appare nella sua descrizione. Avere un vocabolario globale garantisce che tutti i vettori siano **confrontabili** tra loro.
3. Addestriamo un **modello KNN** per utente, iterando su ogni utente
4. Selezioniamo **solo gli item** che l'utente ha **già valutato**, instaziamo un training set e un test set per valutare il modello
5. Con **GridSearch** troviamo gli **iperparametri ottimali** e calcoliamo **l'errore** per controllare la qualità della predizione
6. **Salviamo il modello** in cache per poterlo usare quando vogliamo

Si generano le predizioni solo per item **non ancora valutati** dall'utente, ordinando quelli con rating predetto più alto.

## Risultato BoW

Essendo i modelli creati per ogni utente dobbiamo calcolare la media delle metriche.

|            |      |
|------------|------|
| MSE medio  | 1.09 |
| RMSE medio | 1.04 |

Modelli valutati con successo per 354 utenti su 491. Questo perché alcuni utenti potrebbero avere poche recensioni per allenare per bene il modello e quindi vengono scartati.

## Transformers

I modelli Transformers sono **architetture** di deep learning che processano il testo **considerando** il **contesto** di ogni parola. Generano **embedding** densi, cioè vettori a **dimensione fissa** che catturano il **significato semantico e contestuale** del testo: ad esempio, prodotti con descrizioni semanticamente simili (es. "chitarra acustica a 6 corde" e "sei corde per folk") avranno vettori molto vicini nello spazio, anche se usano parole diverse.

### Implementazione

Utilizziamo il modello: [sentence-transformers/all-MiniLM-L6-v2](#) (non è stato fatto vedere a lezione ma abbiamo deciso di usarlo per la sua velocità e qualità di computazione).

Il codice prima calcola gli **embedding** per **ogni item** unico del catalogo. Per ciascuno, prende il suo testo e lo passa al modello Transformer. Il modello restituisce un embedding e la funzione `mean_pooling` aggrega le informazioni.

### Mean pooling

Il Transformer produce un embedding per ogni token nel testo. Il "**mean pooling**" è una tecnica per **aggregare** questi vettori in un **unico** embedding che rappresenta l'intero testo, calcolando la **media** dei vettori dei token validi (ignorando il padding). Viene fatto perché è l'unico formato di 'input' che modelli come KNN accettano.

## Addestramento e ottimizzazione

I transformers funzionano diversamente rispetto ai BoW:

**PRIMA** che inizi il ciclo sugli utenti:

- Si **prendono** i **testi** di TUTTI gli articoli del catalogo, una sola volta.
- Si usa il modello Transformer per **generare** i vettori di **embedding** per ogni articolo. Questo è un processo lento e pesante, ma viene fatto una sola volta all'inizio.
- Questi embedding vengono **salvati** in una lista/dizionario che associa ogni itemID al suo vettore di embedding.

All'**interno** del ciclo per l'utente della **funzione di raccomandazione**:

- Si prendono gli **ID** degli **articoli** che "X" ha valutato.
- Invece di calcolare qualcosa, si va semplicemente a **recuperare/cercare** i vettori di embedding già pronti dalla mappa creata in precedenza.
- Il modello **KNN** viene **addestrato** su questi vettori pre-esistenti.

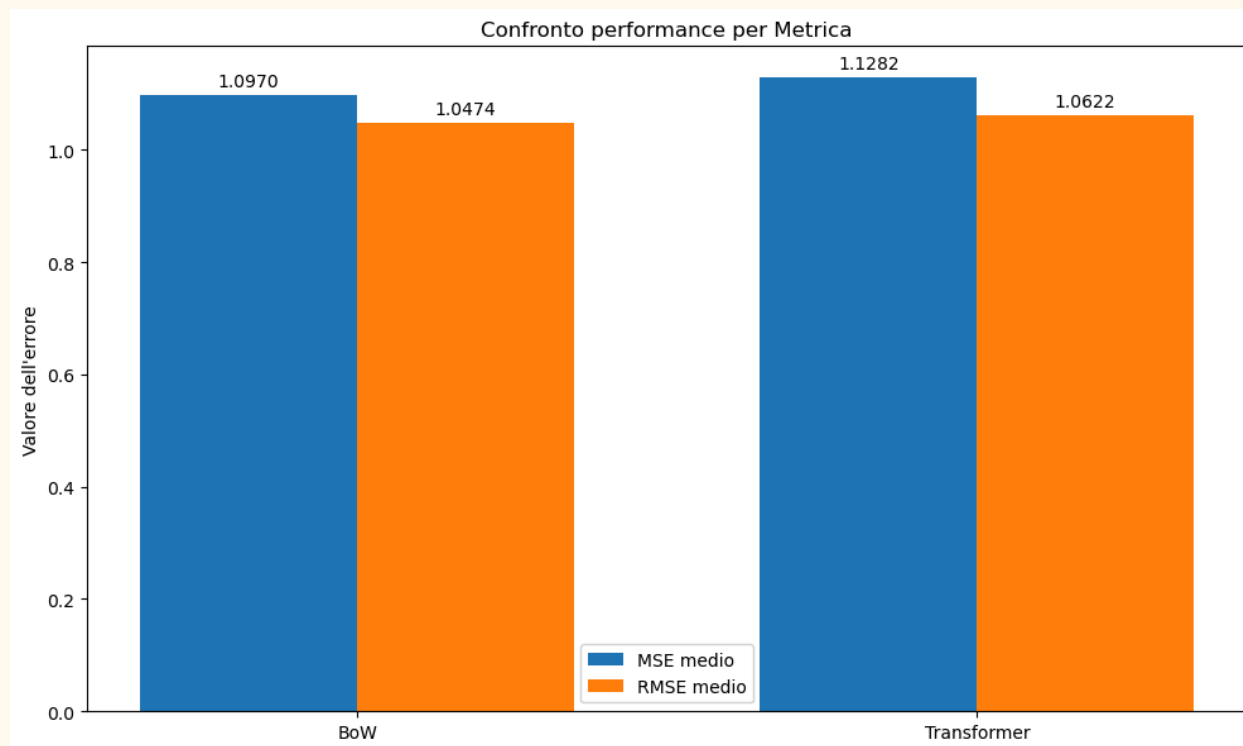
Inoltre, esattamente come per il BoW, viene utilizzata **GridSearchCV** per trovare la combinazione ottimale di `n_neighbors` e `metric` per il modello **KNeighborsRegressor**.

## Risultati

|            |      |
|------------|------|
| MSE medio  | 1.12 |
| RMSE medio | 1.06 |

Modelli valutati con successo per 354 utenti su 491.

## Confronto dei Risultati

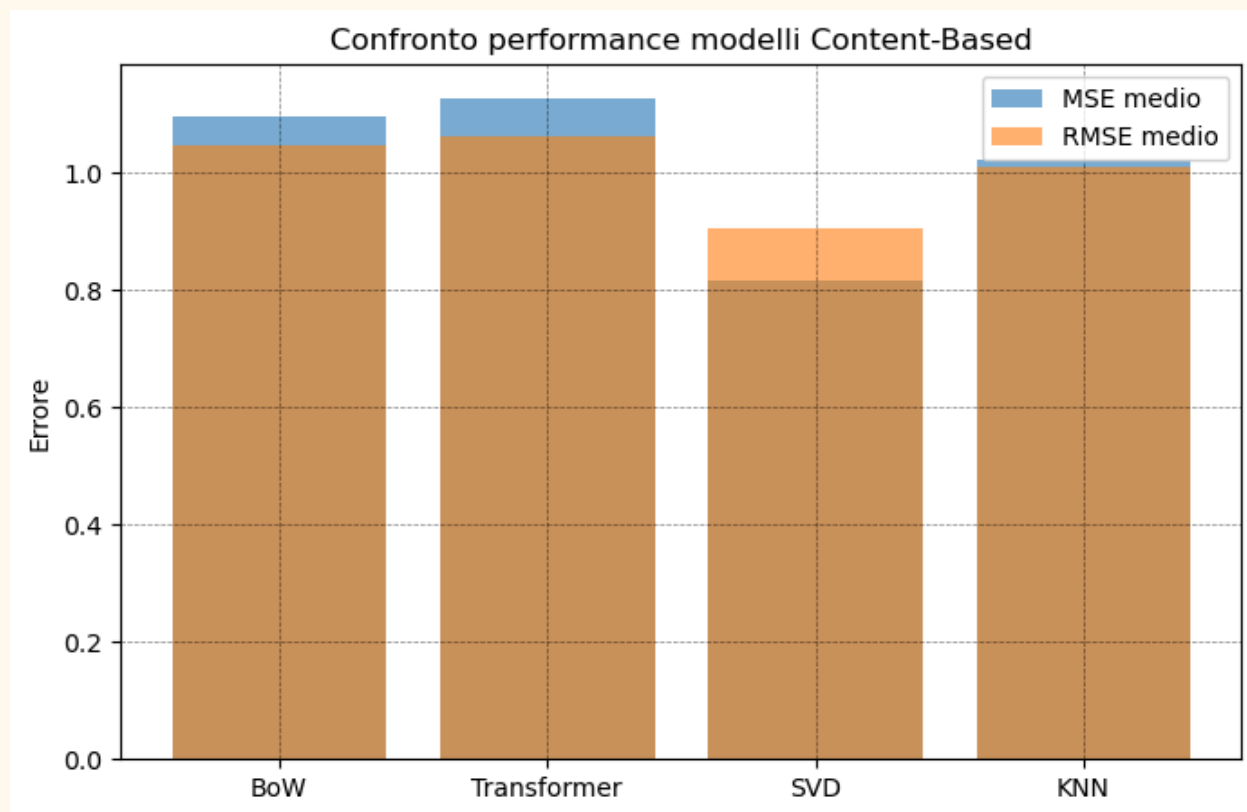


Il modello basato su **Bag-of-Words (BoW)** ha ottenuto un errore medio (sia MSE che RMSE) **leggermente inferiore** rispetto al modello basato su Transformer.

In teoria, ci si aspetterebbe che il modello Transformer, grazie alla sua comprensione semantica, superi il più semplice Bag-of-Words (BoW). Tuttavia, il grafico mostra il contrario: **il BoW ha un errore di predizione leggermente inferiore.**

Una spiegazione potrebbe essere che **KNN** è un modello **semplice** basato sulla distanza. Potrebbe **non** essere in grado di **sfruttare appieno** la ricchezza e la complessità degli embedding Transformer, specialmente con **pochi dati** di training per utente (la "maledizione della dimensionalità")

Un secondo grafico confronta questi modelli content-based con i risultati di modelli di **Collaborative Filtering** (SVD, KNNBasic), che si basano sui pattern di voto degli utenti invece che sul contenuto dei prodotti.



Il confronto con SVD (un modello di Collaborative Filtering) mostra che quest'ultimo ottiene **performance migliori** (MSE più basso). Questo suggerisce che la tecnica di riduzione dimensionale SVD (Singular Value Decomposition) è stata la più **efficace** nel **catturare** le informazioni utili dal contenuto per predire i rating.

## PROGETTO AVANZATO

L'obiettivo è **classificare il sentiment** delle recensioni usando il modello BoW e modello Transformer. Confrontare infine le performance dei 2 modelli.

### Manipolazione dei dati

Selezioniamo solo le colonne rilevanti (**title, text, rating**) dove le prime 2 servono a creare il vettore per la recensione e il 'rating' servirà ad assegnare il '**tipo**' di **recensione** (negativa, neutra, positiva).

Riusciamo a **mappare il 'sentiment' con i 'rating'** andando a etichettare come

- **negative** (0) tutte le recensioni con una valutazione minore di 2
- **neutre** (1) tutte le recensioni con valutazione uguale a 2
- **positive** (2) tutte le recensioni con valutazione maggiore di 2.

Infine estraiamo a caso **10000 campioni** per addestrare e misurare i nostri due modelli. Questa scelta è sicuramente diversa rispetto alle precedenti, dove si era filtrato il dataframe in base al numero di recensioni.

Il motivo è che essendo questa una sentiment analysis il modello viene **addestrato** frase per frase (o "recensione per recensione"). Il **risultato finale** è un modello che data una frase riesce a capire se è negativa, neutra o positiva. Deve dunque ricevere in input una frase sola, e non c'è bisogno che chiunque abbia scritto la recensione, ne abbia fatte molte altre.

L'unico accorgimento è cercare di rendere le **classi di sentiment** (negativo, neutro, positivo) tutte con la **stessa proporzione**. Infatti ci possiamo subito accorgere che le classi con un sentiment positivo sono circa 2.4 milioni mentre le classi con sentiment negativo e neutro combinate sono circa 600 mila.

|                                 |         |
|---------------------------------|---------|
| <b>Num. Recensioni Negative</b> | 395708  |
| <b>Num. Recensioni Neutre</b>   | 197080  |
| <b>Num. Recensioni Positive</b> | 2423610 |

Scegliere completamente a caso il campione porterebbe (per pura statistica) ad avere un campione pieno di recensioni positive, e quindi allenarlo su quelle e creare un bias nel modello.

Questa è la **distribuzione** di *df\_sample* che rappresenta il campione di 10000 unità.

|                          |     |
|--------------------------|-----|
| Num. Recensioni Negative | 34% |
| Num. Recensioni Neutre   | 33% |
| Num. Recensioni Positive | 32% |

## Pre-Processing e Suddivisione dati

Come abbiamo visto nel progetto intermedio, dobbiamo inizialmente ‘trasformare’ le frasi e il testo delle recensioni in un formato adatto ai nostri modelli. L’implementazione non si discosta da quella vista nel progetto intermedio.

Infine dividiamo i dati in un **training set** composto dal 75% dai 10000 campioni estratti casualmente e un **test set** del 25%.

Essendo che il campione è stato estratto casualmente dobbiamo fare uso di ‘stratify = y’. Questo **comando** ci assicura che la **proporzione** di recensioni negative, neutre e positive sia **uguale** nel training set e nel test set. Senza questo comando c’è il rischio di allenare un modello con un’alta proporzione di recensioni di un certo tipo (magari tutte negative) e quando poi lo si va a testare sul test set (che magari ha un’alta proporzione di recensioni positive) si otterrebbe un modello poco performante.

## Bag of Words

Ora comincia il vero cambiamento rispetto al progetto intermedio. Il nostro modello Bag of Words identifica le 5000 parole più comuni/frequenti. Queste diventano il vocabolario globale del nostro modello ed ogni vettore sarà composto da 5000 elementi. Quello che facciamo è allenare il modello facendogli **imparare l’associazione** di un sentiment (negativo, neutro, positivo) **con un gruppo di vettori**.

## Grid Search

Per coerenza con le altre parti del progetto abbiamo deciso di non usare un algoritmo di classificazione con i suoi iperparametri standard ma abbiamo cercato i suoi **iperparametri ottimali**.

## Support Vector Machine

Invece che usare il KNN o SVD abbiamo deciso di usare **SVM**. Questo algoritmo di classificazione è stato accennato in laboratorio e dopo alcuni test abbiamo osservato la sua accuratezza e la sua velocità nel processare i dati, con **risultati** decisamente **migliori** rispetto agli altri algoritmi.

L'algoritmo usa **2 parametri**:

- **C** = che controlla quanto deve essere **precisa** la separazione tra le classi, più è basso il punteggio più il modello è "tollerante". Ma anche un valore troppo alto rischia di ottenere degli overfitting. Testiamo C su questi parametri **0.1 - 1 - 10 - 100**
- **Dual** = si riferisce ai due diversi problemi matematici (**primo e duale**) che possono essere risolti per trovare l'iperpiano.

Ad ogni modo usiamo LinearSVC per applicare l'algoritmo SVC, usando i parametri C = 0.1 e Dual = True. Questo è quello che succede:

- **Rappresentiamo** ogni **recensione** come un **punto** in uno spazio a 5000 dimensioni
- **Troviamo l'iperpiano** (il piano separato dalla retta) che massimizza il margine tra le classi di sentiment (negativo, neutro, positivo).
- Usiamo il **parametro C** trovato da GridSearch per **bilanciare** correttamente la larghezza del margine, ovvero la sua **tolleranza agli errori**.

## Risultati Bag of Words

Il modello ha raggiunto un'accuratezza del 69%. Sembra un'accuratezza bassa, ma è stabile per tutte le classi.

|                 | Precision | Recall | F1 Score | Istanze per ogni classe |
|-----------------|-----------|--------|----------|-------------------------|
| <b>Negativo</b> | 0.70      | 0.72   | 0.71     | 855                     |
| <b>Neutro</b>   | 0.61      | 0.53   | 0.57     | 812                     |
| <b>Positivo</b> | 0.76      | 0.83   | 0.79     | 833                     |

Le **performance** del modello sono infatti ben **proporzionate**. Le recensioni **neutre** sono le **più difficili da predire** con un F1 score di 0.57, ma è comunque un risultato accettabile.



## Transformer

Anche per questo modello non ci siamo discostati troppo da quello che avevamo fatto nel progetto intermedio. Ovviamente non abbiamo dovuto creare nessun profilo utente ma solo addestrare il modello a predire la classe/tipo di recensione.

Come per BoW abbiamo trovato gli iperparametri migliori con **GridSearch: C = 1, Dual = True**

Ciò significa che rispetto a prima il modello è **meno tollerante**, perché probabilmente la suddivisione in classi è più “pulita”.

## Support Vector Machine

L'unica differenza con BoW è che invece che avere un vettore lungo 5000 **il vettore è lungo 384**.

Questo perché all-MiniLM-L6-v2 è stato **pre-addestrato** su una quantità enorme di testo per imparare il significato contestuale delle frasi. Quando riceve in input una recensione, viene riassunta in un **vettore denso** di una **dimensione fissa**. Questa dimensione è tendenzialmente decisa dai creatori del modello.

## Risultati del Modello Transformer

Il modello ha raggiunto **un'accuratezza del 67%**. Leggermente più bassa di BoW.

|                 | Precision | Recall | F1 Score | Istanze per ogni classe |
|-----------------|-----------|--------|----------|-------------------------|
| <b>Negativo</b> | 0.68      | 0.74   | 0.71     | 855                     |
| <b>Neutro</b>   | 0.59      | 0.49   | 0.54     | 812                     |
| <b>Positivo</b> | 0.73      | 0.79   | 0.76     | 833                     |

Il **modello non fornisce risultati troppo diversi** da quelli forniti da BoW:

- BoW Accuracy → 0.69
- Transformer Accuracy → 0.67

## CONCLUSIONI

Il confronto tra i sistemi di raccomandazione ha rivelato che l'approccio basato sul Collaborative Filtering, e in particolare il modello SVD, si è dimostrato il più performante, con una netta superiorità del suo errore medio (RMSE di 0.90) rispetto a tutti gli altri metodi Content-Based.

Questo ci porta alla seconda conclusione chiave del progetto, emersa sia nel "Progetto Intermedio" che in quello "Avanzato". In entrambe le fasi, il modello classico Bag-of-Words ha ottenuto performance leggermente migliori (1.04 di RMSE e 0.69 di Accuracy) rispetto a una sofisticata architettura Transformer (1.06 di RMSE e 0.67 di Accuracy).

Questi risultati non indicano affatto un inferiorità della tecnologia Transformer, ma è la conseguenza di un tentativo di creare un confronto equo, i Transformer sono stati "castrati" per essere compatibili con i modelli a valle, ovvero KNN e SVM.

Il loro potenziale è stato limitato al ruolo di semplice estrattori di embedding: infatti si è dovuto applicare il mean pooling per condensare l'output in un singolo vettore, in quanto era l'unico formato di 'input' che modelli come KNN accettavano.

Questo approccio ha impedito il processo fondamentale di fine-tuning, dove il Transformer avrebbe dovuto adattare i suoi pesi interni al nuovo compito. Di conseguenza, la ricchezza semantica contenuta in un vettore denso è andata praticamente persa, a differenza della rappresentazione più diretta di BoW, che si è rivelata più "digeribile" per i semplici modelli geometrici utilizzati.